

Executive Summary

Questo documento definisce in modo chiaro scopo, obiettivi e funzionalità principali del **vertical slice 2v2** di *RefactorTactics*, sviluppato in Unreal Engine 5.6 su mappa esagonale 【13†L1495-L1502】. Il PRD organizza e allinea il team di design e sviluppo sui requisiti, come indicato da Atlassian: scopo, funzionalità chiave e criteri di successo devono essere una sola fonte di verità 【13†L1529-L1532】. Abbiamo quindi descritto tutte le meccaniche di gioco (azioni, fasi di risoluzione, movimento, targeting, fallback, reazioni, stati, terreni, coperture, equipaggiamento, eroi e loadout), oltre ai requisiti tecnici (determinismo, performance, networking) e modello dati. L'implementazione è suddivisa in milestone progressive (dal setup all'integrazione finale), affiancata da strumenti di debug. Il documento include inoltre esempi di JSON per *PrimaryDataAsset*, diagrammi mermaid (timeline e ER), e matrici di test manuali/automatici per la convalida del vertical slice. Tutti i requisiti sono espressi in termini concreti, e ove manchino dettagli espliciti abbiamo indicato "non specificato" secondo le linee guida.

1. Obiettivi di Prodotto e Scope

- **Ambito di gioco:** *RefactorTactics* è un gioco strategico a turni su **griglia esagonale**, 2 squadre da 2 eroi (2v2). Il vertical slice comprende tutte le meccaniche base di combattimento tattico, senza livelli di progressione o contenuti casuali.
- **Motivazione:** Validare il concept di gameplay (movimento, combattimento, abilità, ambientazione) in un prototipo giocabile. Dimostrare a stakeholder e sviluppatori che i sistemi funzionano insieme in modo divertente ed efficiente 【13†L1529-L1532】.
- **Tecnologia:** Unreal Engine 5.6 (C++) con approccio autoritario server-side per sincronizzazione. Il sistema deve essere deterministico (stessa simulazione su server e client) tramite seed fissi per i generatori di numeri casuali 【8†L27-L34】.
- **Obiettivi chiave:** Implementare il "nucleo" (core) con 4 eroi distinti, 6-8 azioni per eroe, tipi di terreno e coperture, equipaggiamenti iniziali, regole di collisione e interazioni ambientali. Tutte le funzionalità devono essere testabili e riproducibili (replay deterministico) 【8†L27-L34】 【15†L115-L123】.
- **Esclusioni:** Non si include IA avanzata né campagne narrative. Non ci sono upgrade o progressione tra partite. *Out of scope:* multiplayer massivo, grafica avanzata, editor di mappe dinamico.
- **Stakeholder:** Team di design (impostazione regole e bilanciamento), team di engine (implementazione UE5, networking), QA (test manuale/automatizzato), producer.
- **Criteri di successo (DoD):** Il gioco è uno slice giocabile completo: tutte le azioni hanno ID e parametri stabili, i terreni e le coperture funzionano secondo specifiche, le collisioni simultanee sono regolate, la propagazione di effetti ambientali è deterministica. In particolare, la sequenza di gioco ripetuta con lo stesso seed deve dare sempre lo stesso risultato 【8†L27-L34】 【15†L115-L123】.

2. Requisiti Funzionali

2.1 Azioni Gioco

- **Tipi di azioni:** Il sistema include le azioni fondamentali (es. *Move*, *BasicAttack*, *Guard*, *Wait*, *Activate*, *Interact*), azioni di movimento speciali (*Sprint*, *Dash*, *Charge*, *Leap*, *Reposition*), azioni offensive (*PrecisionAttack*, *HeavyAttack*, *LineAttack*, *CircularAoE*, *SuppressiveLine*, *MarkTarget*), azioni difensive/reazione (*Counter*, *Intercept*, *Deflect*, *Brace*, *Shield*, *Cleanse*) e azioni di supporto/ambiente (*Heal*, *CreateWater*, *Ignite*, *Electrify*, *CreateCover*, *ModifyArc*). Ogni azione è definita da un ID univoco, fase di risoluzione, priorità, portata, costo in risorse (es. punti movimento, MP) e durata/cooldown.
- **Azioni selezionabili:** All'inizio del turno ogni eroe sceglie un percorso di movimento (se *Move*) e un'Azione Principale, oltre a eventuali reazioni preparate. Come tabella riassuntiva, l'azione base *Move* (MP standard 5) consuma tempo nella fase 20 con priorità 50 ed è fallback 'Stop' se bloccata. *Attack base* dipende dall'eroe e arma e viene risolta in fase 40 con priorità 50. *Guard* ha fase 10 (preparation) e riduce il prossimo danno di 15, resiste a uno spinta di 1. *Wait* (fase 20, priorità 100) non fa nulla e può solo impostare facing o preparare reazioni.

ID Azione	Tipo	Fase	Priorità	Portata / Costo	Descrizione
Action.Wait	Fondamentale	20	100	—	Nessuna azione; solo facing/reazione
Action.Move	Movimento	20	50	5 MP	Movimento su percorso adiacente
Action.BasicAttack	Offensive	40	50	1 cella (CC)	Attacco base (danno variabile per eroe)
Action.Guard	Difensiva	10	40	Self (buff)	Riduce primo danno di 15, blocca push 1
Action.Activate	Principale	40	70	1	Attiva oggetto/obiettivo adiacente
Action.Interact	Principale	40	80	1	Interagisce (es. addobbare gadget)
...	...	...	...	...	Altre azioni: Sprint, Dash, ecc.

(Dettagli completi per ciascuna azione sono definiti nel catalogo di bilanciamento; qui si ribadiscono i più critici).

2.2 Fasi di Risoluzione

La partita è a turni simultanei (planning + resolution). Ad ogni turno si segue la sequenza di fasi **fisse**:
 | Fase (Nome) | Codice | Contenuto principale |
 |-----|-----|-----| | **0. Snapshot** | 0 | Congelamento dello stato iniziale, intenti e semi RNG | | **1. Preparazione** | 10 | Applicazione scudi/Stance, trappole, setup reazioni | | **2. Movimento** | 20 | Movimento simultaneo (micro-step) | | **3. Controllo** | 30 | Root/Push/

Interrupt/Interposizione | | **4. Attacco** | 40 | Risoluzione attacchi ed abilità offensive e cure | | **5. Ambiente** | 50 | Trigger hazard (acqua, fuoco, elettricità, fumo) | | **6. Cleanup** | 60 | KO, verifica obiettivi, decremento cooldown, log |

L'ordine interno rispetta fase \priorità\ID (in modo stabile), evitando dipendenze non determinate da id statici o dall'ordine di un TMap 【11†L29-L37】 【15†L115-L123】. Tutte le azioni dichiarano fase e priorità: priorità minore significa risolto prima nella stessa fase. Per esempio, *Dash* (fase 20, priorità 30) si risolve prima di *Move* (20, priorità 50).

2.3 Regole di Movimento

- **Budget:** 5 MP standard. Costo per cella: 1 MP (terreno normale), 2 MP (terreno difficile o salire di un livello con rampa). *Sprint* consuma 8 MP totali (compenso Move+Azione), *Reposition* e *Dash* hanno portata fissa (2 e 3 celle).
- **Vincoli:** Non si può attraversare una cella occupata da unità solida; nemmeno termina in cella occupata. Il percorso viene scelto all'inizio del turno e non è ricalcolato durante la resolution.
- **Fallback Movimento:** Se il percorso si blocca (ad es. unità o ostacolo nuovo), l'unità si ferma nell'ultima cella valida (comportamento *Fallback.Stop*) 【11†L29-L37】. Non sono previste azioni di bypass automatico; il giocatore deve pianificare diversamente.
- **Collisioni Multiple:** In caso di conflitto (es. 2 unità puntano a stessa cella), le regole decise sono:
 - Se pari priorità/MP, entrambe si fermano prima della cella contesa.
 - Se un'unità fa *Charge* e l'altra *Move*, *Charge* prevale: l'unità in movimento si ferma indietro.
 - Se una cella è già occupata da un'unità ferma, chi entra si ferma prima.
 - Due *Charge* opposte si fermano entrambe. (Vedere schema di regolazione collisioni in *Catalogo Balancing v0.1*).
- **Movimenti Speciali:** *Dash* e *Charge* ignorano il normale percorso Move (movimento lineare); *Leap* salta unità e coperture basse; *Push/Pull* (come azione di controllo) sposta target di 1 cella se libera. Coperture alte o porte chiuse bloccano spostamenti lineari e subentrano regole di interruzione.

2.4 Targeting e Forme

- Ogni azione definisce un tipo di targeting (bersaglio singolo, cella, area, linea, cone, ecc.). Esempi: *LineAttack* colpisce la prima unità in una direzione esagonale (range 5), *CircularAoE* prende come centro una cella scelta entro 4 caselle.
- Esistono forme standard:
 - **Self:** bersaglio se stessi (es. Guard, Shield).
 - **Cell/Point:** selezione di una cella nel range (es. Ignite sul terreno).
 - **Direzione:** linea in una delle 6 direzioni (es. LineAttack, Charge).
 - **Area:** raggio di celle attorno a un punto (es. AoE raggio 1).
- Il **targeting** verifica collisioni con ostacoli (coperture alte bloccano linee, coperture basse riducono danno se da dietro) e visibilità (LOS, fumo riduce visibilità a 2).

2.5 Fallback delle Azioni

Ogni azione principale (dash, attacchi, AoE, cura, ecc.) dichiara un comportamento di fallback se il bersaglio non è valido al momento dell'effettiva risoluzione: ad esempio *BasicAttack* può usare

`Fallback.BasicAttack` (cerca nuovo bersaglio vicino) o `Fallback.Cancel` (nulla). Per il vertical slice:

- *Move*: sempre `Fallback.Stop`.
- *AoE*: `Fallback.AttackCell` (colpisce comunque la cella pianificata).
- *Attacchi diretti e cure*: `Fallback.Cancel` (nulla).
- *Reazioni*: non hanno fallback (si attivano o non succede nulla).

Questo assicura comportamenti prevedibili in caso di movimento dei bersagli o modifiche di piano.

2.6 Reazioni e Stati

- **Reazioni**: Azioni che scattano su trigger durante il turno altrui. Es. *Counter* (contrattacco da 16d dopo essere colpiti), *Intercept* (prende l'attacco diretto destinato a un alleato entro 2 celle), *Deflect* (riduce 20 danni del primo attacco), *FlowReaction* (Reposition dopo un attacco subito), ecc. Le reazioni sono di slot Reazione e spesso una sola attivazione per turno.
- **Stati Temporanei**: Gli eroi e le cellule possono avere stati come **Burning, Wet, Electrified, Obscured (fumo), Rooted, Exposed, Marked**, ecc. I triggering di questi sono dettagliati nel catalogo: p.es. entrare in Fuoco infligge 10 immediati + *Burning* (8 danni Cleanup per 2 turni), entrare in acqua applica *Wet* (annulla *Burning*, aumenta danni elettrici). Gli stati vengono aggiornati nelle fasi appropriate (Prep/Attacco/Ambiente). La reazione *Cleanse* rimuove uno stato alla scelta del giocatore.

2.7 Terreni di Gioco

La mappa esagonale contiene vari tipi di terreno con effetti specifici. Viene fornita la seguente tabella riassuntiva:

ID Terreno	Nome	Costo MP	Movimento	LOS	Effetto Principale
<code>Terrain.Floor</code>	Pavimento	1	Normale	Libero	-
<code>Terrain.Rough</code>	Accidentato	2	Sì (no Dash)	Libero	Rallenta (costo+2 MP)
<code>Terrain.ShallowWater</code>	Acqua bassa	2	Normale	Libero	Applica <i>Wet</i> , conduce elettricità, spegne <i>Burning</i>
<code>Terrain.Fire</code>	Fuoco	2	Normale	Parziale	Dà 10 danni + <i>Burning</i>
<code>Terrain.Conductive</code>	Metallica	1	Normale	Libero	Conduce elettricità
<code>Terrain.Smoke</code>	Fumo	1	Normale	Ridotta	Obscured (riduce visibilità)

ID Terreno	Nome	Costo MP	Movimento	LOS	Effetto Principale
<code>Terrain.Ice</code>	Ghiaccio	1	<i>Scivoloso</i> ¹	Libero	Se entri con ≥ 2 MP scivoli una cella
<code>Terrain.HighGround</code>	Quota alta	1	Dipende dagli archi	Libero	Bonus visivo

1. *Scivoloso*: se un'unità entra con ≥ 2 MP rimaste, scivola di 1 cella nella direzione di ingresso (sospensibile per test iniziale).

Ogni terreno può interagire con abilità ambientali: es. l'acqua può essere gelata o attraversata da elettricità **【8†L27-L34】**, il fuoco può propagarsi su vegetazione, il ghiaccio può essere rotto. La fase *Ambiente (50)* gestisce propagazioni (in ordine di distanza) in modo deterministico.

2.8 Coperture e Strutture

Le coperture forniscono protezione direzionale o totale. Riepilogo degli elementi principali:

ID Struttura	Tipo	Movimento	LOS	Integrità	Protezione
<code>LowCover</code>	Copertura bassa	Blocca proiettile sotto	Parziale (offuscato)	30	Riduce danno 10 da una direzione
<code>HighCover</code>	Copertura alta	Blocca completamente	Bloccato	50	Protezione totale (0 danni oltre)
<code>Structure.Door</code>	Porta	Variabile (dipende stato)	Variabile	40	Variabile (es. 10 se bloccata)
<code>Bridge</code>	Ponte	Permette arco sopra	Libero	40	Nessuna (aria aperta)
<code>KineticPanel</code>	Pannello cinetico	Blocca proiettile basso	Parziale	30	Protegge per 10 danni

- **LowCover**: associata a un bordo di cella, riduce di 10 danni frontali, ma da lato diverso è inefficace; non blocca completamente linea di vista.
- **HighCover**: blocca movimento e linea di tiro; assorbe interamente i colpi fino a distruzione.
- **Porta**: può essere *Open/Closed/Locked/Destroyed*. Cambio di stato aggiorna il grafo di connessione (es. una porta chiusa blocca il cammino e la LOS).
- **Ponte (Bridge)**: collegamento tra due celle. Se disattivo o distrutto, non è attraversabile.
- **Pannello cinetico**: gadget di Bastion, crea una copertura temporanea con specifiche.

Le coperture interagiscono con abilità: es. *CreateCover* posiziona un *LowCover*; *Breccia/Gadget* possono danneggiare strutture (colpo, granata, *BreachCharge*); *LineAttack* o *Charge* possono essere interrotte da cover alte.

2.9 Equipaggiamenti

Ogni eroe inizia la partita configurato con: **1 Variante d'Arma, 1 Gadget, 1 Modulo Reazione** (non si trovano o ottengono durante il turno). Non ci sono livelli o upgrade in partita.

- **Varianti di Arma:** modificano l'attacco base dell'eroe. Ad esempio:

ID Variant Vantaggio Svantaggio	----- ----- -----	
Weapon.Precision +1 range -4 danni		Weapon.Impact Push 1 -1 range
Weapon.Overcharge +6 danni +1 cooldown		Weapon.Split +1 bersaglio -6 danni
Weapon.Suppressive Applica Slow -5 danni		Weapon.Environmental Potenza hazard -5 danni

- **Gadget:** elementi d'uso singolo con effetto attivo, es. *Medkit* (cura 18), *Sprinkler* (crea acqua), *SmokeEmitter* (crea fumo), *Anchor* (blocca push), ecc. Ogni gadget ha un cooldown in turni. (Vedi tabella dettagliata nel catalogo).
- **Moduli Reazione:** potenziamenti passivi che scattano su trigger. Esempi: *Reaction.CounterShot* (contrattacco 14d quando colpiti), *Reaction.ReactiveShield* (+15 scudo quando subisci danno), *Reaction.AllyIntercept* (intercetta attacco alleato), ecc. Ogni modulo si attiva al massimo una volta per turno.

In fase di planning il giocatore sceglie la combinazione Weapon/Gadget/Reaction; un modulo avversario pre-assegnato può essere noto o meno (parte del loadout visibile pre-partita).

2.10 Eroi e Loadout Iniziali

Abbiamo definito **4 eroi** base con ruoli distinti. Tabella riepilogativa (HP, movimento, affinità):

Eroe	Ruolo	Salute	Movimento	Range Visivo	Affinità	Note
Flux	Attacco/ Controllo	90	5	6	Elettricità	Stun/Effetti + elettricità
Riva	Supporto/ Controllo	95	5	5	Acqua	Debuff nemici (Wet) e cura
Bastion	Difesa/Area/ Protezione	120	4	5	Strutture	Pannelli, scudi
Vektor	Assalto/Duello	100	6	6	Movimento	Reazioni per punire avversari

Ogni eroe ha 4 abilità fondamentali uniche (vedi catalogo per effetti), di cui una variante potenziabile. L'**equipaggiamento iniziale consigliato** (WeaponVar, Gadget, Reazione) è: Flux con Precisione+Isolante+Scudo Reattivo, Riva con Cura+Sprinkler+Fuga hazard, Bastion con Pannello Adattivo+Copertura Portatile+Interposizione, Vektor con Intercetto Esteso+Sensore+Dash d'emergenza. Il loadout determina tattiche come push+hazard e combo acqua/elettrico.

3. Requisiti Non Funzionali

- **Determinismo:** Tutta la simulazione deve essere deterministica dato uno stesso seed iniziale. Si adotterà un approccio lockstep/client-server: il server calcola e invia eventi essenziali, i client riflettono la scena. I generatori di numeri casuali (FRandomStream) usano semi sincronizzati replicando un seed in fase di Snapshot [8†L27-L34] . Questo garantisce che muovendo l'input (turni, azioni) con lo stesso seme si ottengano identici risultati. (Nota: Unreal non è deterministico per default a causa di virgola mobile e tick asincroni [15†L115-L123] ; si seguirà tick a step fisso o simile, e si eviteranno funzioni non deterministiche).
- **Performance:** Il gioco deve girare fluido con minime risorse. Si minimizzerà il carico di rete disabilitando la *replicazione* degli attori non necessari (`SetReplicates(false)`) e riducendo il `NetUpdateFrequency` [21†L19-L26] . Si useranno tipi di rete quantizzati (es. `FVector_NetQuantize`) per ridurre banda [21†L34-L41] . I comandi di debug (vtune, profiling) saranno impiegati per individuare colli di bottiglia, e il codice C++ sarà ottimizzato (nessun uso eccessivo di float costosi).
- **Networking:** Topologia client-server con server autoritario. Tutti i dati di gioco (posizioni, stati, HP) vengono replicati dal server ai client. I comandi dei giocatori sono in input e conferme. Si eviterà replicare dati ridondanti: per esempio, le azioni pianificate non sono replicate in realtime (si trasmettono solo intenti finali prima del turno e risultati al Cleanup). È richiesta prevedibilità di risposte anche con lag moderato: si implementeranno rollback limitati, e si sincronizzerà lo stato intermittente tramite checkpoint/semi.
- **Seed/Replay:** Il sistema deve supportare il *salvataggio del seed* di ogni turno per poter riprodurre l'esecuzione (deterministic replay). Durante il debugging si può utilizzare `rt.Debug.DumpSnapshot/TurnLog` e `rt.Debug.VerifyReplay` per confrontare ripetizioni. Il *TurnLog* deve registrare tutti gli eventi del turno (input, collisioni, esiti abilità) in modo che, con lo stesso seed e snapshot, si ottenga identico risultato ad ogni esecuzione automatica [8†L27-L34] [15†L115-L123] .

4. API / Modello Dati Proposto

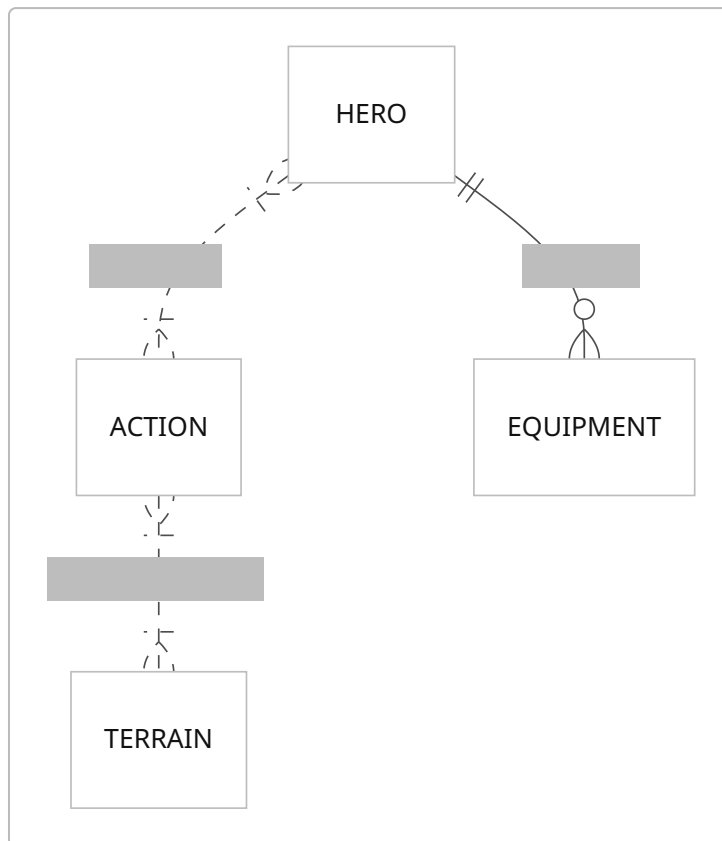
Per flessibilità e caricamento dati ottimale, useremo **PrimaryDataAsset** in C++/Blueprint per definire Actions, Terrains, Eroi ed Equip. Ogni asset ha un ID primario univoco e campi con `UPROPERTY(EditAnywhere)` [1†L72-L81] . Di seguito un modello minimo (esempi di campi):

ActionDataAsset (USTRUCT)	Tipo	Descrizione
ActionId (FName)	stringa	ID univoco (es. "Action.Move")
Phase (uint8)	intero	Fase di risoluzione (es. 20)
Priority (int)	intero	Priorità interna
Range (int)	intero	Portata dell'azione (celle)
Cost (int)	intero	Costo in risorse (es. MP)
Cooldown (int)	intero	Turni di ricarica
Effects (TArray<Effect>)	struct	Effetti applicati (danno, status)

TerrainDataAsset	Tipo	Descrizione
TerrainId (FName)	stringa	ID (es. "Terrain.Fire")
TraversalCost (int)	intero	Costo di movimento della cella
BlocksVision (bool)	bool	Se blocca completamente la LOS
HazardTags (array<FName>)	array	Tag di effetto (Wet, Burning, ecc.)
HeroDataAsset	Tipo	Descrizione
HeroId (FName)	stringa	ID eroe (es. "Hero.Flux")
Health (int)	intero	Salute massima
MoveSpeed (int)	intero	Punti movimento base
SightRange (int)	intero	Raggio visivo
WeaponVariantId (FName)	stringa	Variante d'arma assegnata
GadgetId (FName)	stringa	Gadget assegnato
ReactionModuleId (FName)	stringa	Modulo reazione assegnato

Questo schema dati ci consente di caricare in runtime via Asset Manager (grazie a *PrimaryDataAsset*) le definizioni di ogni entità 【1†L72-L81】. I campi possono essere estesi (ad esempio aggiungere `Damage`, `AreaShape`, `StatusEffect`), mantenendo ID costanti.

Diagramma ER (Entità/Relazioni): Le entità principali e relazioni possono essere rappresentate come segue:



(In pratica: un eroe può eseguire molte azioni; un eroe possiede vari equipaggiamenti (arma, gadget, moduli); un'azione può interessare uno o più terreni).

4.1 Esempi JSON di PrimaryDataAsset

Di seguito alcuni esempi semplificati dei dati asset salvati come JSON (solo campi minimi):

```

// Esempi di Action DataAsset
{
  "ActionId": "Action.Move",
  "Phase": 20,
  "Priority": 50,
  "Range": 0,
  "Cost": 5,
  "Cooldown": 0,
  "Effects": []
}
{
  "ActionId": "Action.Dash",
  "Phase": 20,
  "Priority": 30,
  "Range": 3,
  "Cost": 0,
  "Cooldown": 1,
  "Effects": []
}

```

```
{
  "ActionId": "Action.BasicAttack",
  "Phase": 40,
  "Priority": 50,
  "Range": 1,
  "Cost": 0,
  "Cooldown": 0,
  "Effects": ["Damage:28"]
}
```

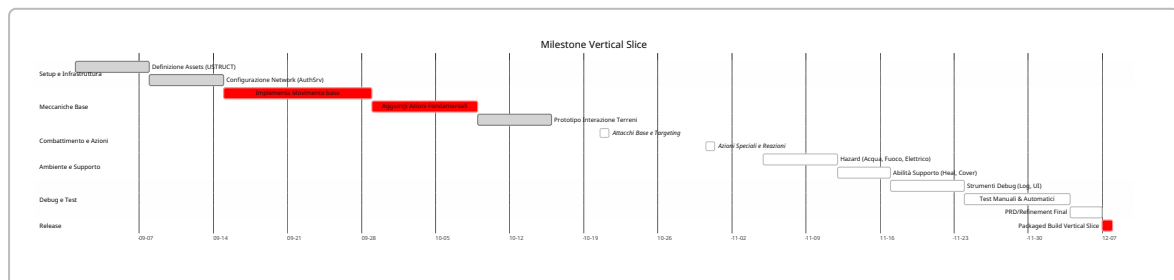
```
// Esempi di Terrain DataAsset
{
  "TerrainId": "Terrain.Floor",
  "TraversalCost": 1,
  "BlocksVision": false
}
{
  "TerrainId": "Terrain.Fire",
  "TraversalCost": 2,
  "BlocksVision": true
}
{
  "TerrainId": "Terrain.ShallowWater",
  "TraversalCost": 2,
  "BlocksVision": false
}
```

```
// Esempi di Hero DataAsset
{
  "HeroId": "Hero.Flux",
  "Health": 90,
  "MoveSpeed": 5,
  "SightRange": 6,
  "WeaponVariantId": "Weapon.Overcharge",
  "GadgetId": "Gadget.Insulator",
  "ReactionModuleId": "Reaction.ReactiveShield"
}
{
  "HeroId": "Hero.Riva",
  "Health": 95,
  "MoveSpeed": 5,
  "SightRange": 5,
  "WeaponVariantId": "Weapon.Precision",
  "GadgetId": "Gadget.Sprinkler",
  "ReactionModuleId": "Reaction.HazardEscape"
}
```

```
// Esempi di Equipment DataAsset (Weapon/Gadget/Reaction)
{
  "WeaponVariantId": "Weapon.Precision",
  "Benefit": "+Range",
  "Drawback": "-Damage"
}
{
  "GadgetId": "Gadget.Medkit",
  "Effect": "Cura 18 HP",
  "Cooldown": 3
}
{
  "ReactionModuleId": "Reaction.CounterShot",
  "Trigger": "Sotto attacco",
  "Effect": "14 danni al nemico"
}
```

5. Flusso di Implementazione e Milestone

Timeline di sviluppo: Il vertical slice sarà realizzato per fasi iterativamente. Ecco una roadmap esemplificativa:



- **Milestone principali:** Finire il motore del movimento e pathfinding, completare tutte le azioni primarie e le interazioni (Milestones *a3-a7*), poi iterare su controlli ambientali e supporto (*a8-a9*). Entro ogni milestone, eseguire test di regressione per garantire determinismo.
- **Debug Tools:** Verranno sviluppati comandi console `rt.Debug` per visualizzare la griglia, cammini, coperture, intenti, e fare dump di snapshot e turn log (come riportato nel catalogo). Questi aiutano a tracciare esattamente come i dati cambiano in ogni fase.

6. Matrice di Test

Test Manuali: Coprono i casi critici delle regole di gioco. Alcuni esempi:

Scenario	Risultato atteso
Due unità tentano stessa cella	Entrambe si fermano (regole collisione)
Move su terreno accidentato	Consuma 2 MP (costo extra)
Dash incontra copertura alta	Si ferma prima (non attraversa ostacolo)

Scenario	Risultato atteso
Push verso cella occupata	Nessuno spostamento illegale (si ferma)
Elettrificazione dell'acqua	Propagazione deterministica (stesso in tutti i test)
Fuoco spento dall'acqua	Terreno "Fuoco" rimosso (brucia non si propaga)
Attacco base su copertura bassa	Danno ridotto di 10 (copertura schermata)
Intercept intercetta attacco alleato	Attaccante mira all'intercettore
AoE colpisce alleato	Si applica danno amico (friendly fire on)
Bersaglio si sposta prima dell'attacco	Viene applicato fallback (es. <i>Cancel</i>)
Porta si chiude durante il turno	Ricostruzione del grafo aggiornata (collision avviene)
Replay stesso turno	TurnLog identico, risultato identico (determinismo)

Test Automatici: Ogni build deve eseguire una suite di test unitari/integrati per validare regole chiave. Per esempio:

Test	Scopo
RT_Move_PathBlocked	Verifica fallback in movimento bloccato
RT_Move_CellConflict	Conflitti di ingresso in cella
RT_Dash_BlockedArc	Dash interrotto da ostacolo
RT_Push_InvalidDestination	Spinta verso cella invalida
RT_Env_WaterElectricPropagation	Propagazione elettrica su acqua
RT_Env_WaterExtinguishesFire	Acqua spegne fuoco
RT_Cover_DirectionalDamageReduction	Danno ridotto da copertura bassa
RT_Reaction_Intercept	Intercept intercetta attacco
RT_Reaction_SingleActivation	Reazione si attiva al massimo 1 volta
RT_Simulation_DeterministicReplay	Replay deterministico e replay identico

Ogni test automatico è replicato più volte con seed fissi per garantire non-variabilità. Questi test vengono eseguiti sia in editor sia nelle build finali. Il checksum finale del mondo simulato deve essere identico in ogni replay 【8†L27-L34】 【15†L115-L123】 .

7. Criteri di Accettazione (Definition of Done)

Il vertical slice è approvato quando:

- **Asset e ID:** Tutte le azioni, terreni, coperture, eroi ecc. hanno ID stabili e dichiarati (es. *Action.Move*). Ogni asset dati (PrimaryDataAsset) ha un getID primario unico.

- **Requisiti funzionali:** Tutti i requisiti sopra elencati sono implementati in modo conforme (movimento, attacchi, reazioni, interazioni ambientali, ecc.). Il comportamento di ogni azione (fase, priorità, targeting, fallback) rispetta le tabelle di design.
- **Regole ambientali:** Ogni terreno e stato applica effetti come specificato. Ad esempio, *Wet* e *Burning* si alternano correttamente con l'acqua/fuoco, e l'elettricità si propaga solo su celle conduttive.
- **Reazioni e collisioni:** Tutte le reazioni sono triggerate correttamente (Counter, Intercept, ecc.) e applicabili una sola volta per turno. Collisioni e push/pull seguono le regole di sezione 2.3 e tabella.
- **Determinismo garantito:** Esiste un TurnLog verificabile che produce risultati identici per replay con stesso seed e snapshot [\[8†L27-L34\]](#) [\[15†L115-L123\]](#) .
- **Performance e stabilità:** La simulazione rimane entro limiti di latenza previsti e senza crash. I test di performance minima passano.
- **Documentazione:** Tutti i file di design (in Docs/Design/Balance) e gli asset vanno creati come da sezione 3 del catalogo. Un commit Git finale includerà `docs: add PRD vertical slice v1.0` con file PRD e qualsiasi asset di esempio.

8. Rischi e Mitigazioni

- **Mancato determinismo UE:** Unreal non è deterministico per default (float, tick asincroni) [\[15†L115-L123\]](#) . *Mitigazione:* utilizzare timestep fisso o controllo esplicito del delta-time; sincronizzare i semi dei RandomStream come discusso [\[8†L27-L34\]](#) ; limitare l'uso di funzioni dipendenti dall'hardware. Se necessario, adottare server autoritario (rollbacks di sicurezza).
- **Lag di rete / Bandwidth:** Replicazione pesante può causare hitches. *Mitigazione:* disattivare la replicazione per attori statici o non essenziali [\[21†L19-L26\]](#) ; usare net quantization (FVector_NetQuantize) per vettori e ridurre frequenza di aggiornamento [\[21†L34-L41\]](#) ; testare su connessioni lente.
- **Ordine di Tick / Stato non sincrono:** L'ordine di ticking UE non è garantito e può variare [\[11†L80-L89\]](#) . *Mitigazione:* usare ordini di ticking espliciti o processare tutti gli attori in elenchi ordinati; evitare dipendenze dall'ordine di un TMap; imporre update deterministico (bEnableEnhancedDeterminism in fisica, se utile).
- **Input dei giocatori:** Race condition tra movimenti. *Mitigazione:* la fase Snapshot congela gli input, e le interazioni (Interrupt, Counter) sono basate su regole fisse. Uso di `EventSequence` stabile per ordinare azioni parallele.
- **Ambiente e Hazard:** Molte interazioni possibili (acqua + elettrico + fuoco). *Mitigazione:* testare tutte le combo possibili con tool di simulazione automatica; UI indicatori di stato (*Wet*, *Burning*, ecc.) per chiarezza utente; log di debug per propagazioni.
- **Crash UI/UX:** Il vertical slice non include UI definitiva. *Mitigazione:* focalizzarsi su logica backend, usare strumenti console di debug (`rt.Debug`) per visualizzare griglia, intenzioni e risolvere bug senza UI finale.

9. File e Commit Proposti

- **Doc di design:** `Docs/Design/PRD_RefactorTactics_v1.0.md` (questo documento in formato Markdown, sezione Agile/PRD).
- **Cataloghi di bilanciamento:** Seguire struttura del catalogo: `Docs/Design/Balance/RT_ActionCatalog_v0.1.md` , `RT_TerrainCatalog_v0.1.md` , `RT_EquipmentCatalog_v0.1.md` , `RT_HeroCatalog_v0.1.md` , ecc.

- **Asset di gioco:** Directory `Content/RefactorTactics/Data/Actions/`, `/Terrains/`, `/Equipment/`, `/Heroes/` per i vari `PrimaryDataAsset`.
- **Codice C++:** Includere `USTRUCT` e `UPrimaryDataAsset` corrispondenti: ad es. `URTActionDataAsset`, `URTTerrainDataAsset`, `URTHeroDataAsset`, `URTEqDataAsset`.
- **Commit Git:** Esempio di messaggio finale:

```
docs: add PRD per vertical slice RefactorTactics (v1.0)
```

Questo commit include il file PRD Markdown, eventuali aggiornamenti ai cataloghi esistenti e placeholder di esempio JSON in `Content/`. Il seguente ticket/product backlog item può riferirsi a questo commit.

Fonti e best practice: Per la stesura di questo PRD ci si è basati su linee guida Atlassian [【13†L1495-L1502】](#) [【13†L1529-L1532】](#) e documentazione Unreal su Data Asset [【1†L72-L81】](#) e rete [【21†L19-L27】](#) [【21†L34-L41】](#), integrando le regole di gioco definite nel catalogo (es. gestione dei semi RNG [【8†L27-L34】](#)) e nei forum tecnici (es. importanza di simulazione deterministica [【15†L115-L123】](#)).
